

CS121 Data Structures in C

12 Sorting - 1

Outline

- Sorting Preliminaries
- Comparison based sorting
 - ♦ Insertion sort
 - Insertion sort (direct)
 - Shell sort
 - ♦ Selection sort
 - Heap sort
 - ♦ Merge sort

Sorting Preliminaries

- Given a list of elements (integers and be legal), sorting it and return a list of elements in the order.
 - ♦ 3 2 4 5 1 7 6 8 9 becomes 1 2 3 4 5 6 7 8 9, assuming the ordering is $\leq$.
- The data starts at position 0 in all sorts.
- The sorting is based on comparison of two elements, so we assume the existence of “<” and “>” operators.

“<” and “>” operators are the only operations allowed on the input data besides the assignment operator.

Sorting under these conditions is known as comparison-based sorting.
- A list can be stored in an array or a linked list.
 - ♦ Sorting is an re-arrangement of the elements
 - For an array, swapping is used.
 - For a linked list, the exchange is implemented by pointers' operations.

Sorting Classifications

- Sorting algorithms are classified as either *internal* or *external*.
- Internal sort (Our focus)
 - ♦ All data are hold in main memory during the sorting process.
- External sort
 - ♦ Sorts cannot be performed in main memory and must be done on disk or tape.

Insertion Sort

Insertion Sort

- Commonly used by human when playing card games
 - 1) Initially $p = 1$
 - 2) Let the first p elements be sorted.
 - 3) Insert the $(p+1)$ th element properly in the list so that now $p+1$ elements are sorted.
 - 4) increment p and go to step (2)



Insertion Sort

Original	34	8	64	51	32	21	Positions Moved
After $p = 1$	8	34	64	51	32	21	1
After $p = 2$	8	34	64	51	32	21	0
After $p = 3$	8	34	51	64	32	21	1
After $p = 4$	8	32	34	51	64	21	3
After $p = 5$	8	21	32	34	51	64	4

see applet

<http://www.cis.upenn.edu/~matuszek/cse121-2003/Applets/Chap03/Insertion/InsertSort.html>

Insertion Sort

```
void
InsertionSort( ElementType A[ ], int N )
{
    int j, P;
    ElementType Tmp;

    for( P = 1; P < N; P++ )
    {
        Tmp = A[ P ];
        for( j = P; j > 0 && A[ j - 1 ] > Tmp; j-- )
            A[ j ] = A[ j - 1 ];
        A[ j ] = Tmp;
    }
}
```


Analysis: inputs are in reverse order (worst case inputs)

```
void
InsertionSort( ElementType A[ ], int N )
{
    int j, P;
    ElementType Tmp;

    for( P = 1; P < N; P++ )
    {
        Tmp = A[ P ];
        for( j = P; j > 0 && A[ j - 1 ] > Tmp; j-- )
            A[ j ] = A[ j - 1 ]; // move
        A[ j ] = Tmp;
    }
}
```

$O(N)$ $O(N)$

- The nested loops
 - ♦ Each loop can take at most $N-1$ iterations, insertion sort runs in $O(N^2)$ time.
- Consider an input in a reverse sorted list (worst case inputs)
 - ♦ When $A[P]$ is inserted into the sorted $A[0..P-1]$, we need to compare $A[P]$ with all elements in $A[0..P-1]$ and move each element one position to the right $\Rightarrow \Omega(P)$ steps.
 - ♦ Inner loop is executed P times, for each $P=1..(N-1)$.
 - ♦ The total number of steps is at least $\Omega(\sum_{i=1}^{N-1} i) = \Omega(N(N-1)/2) = \Omega(N^2)$.
- The bound is tight $\Theta(N^2)$, which is the time complexity for the worst case inputs (worst case time complexity).
- Space requirement is $O(N)$.

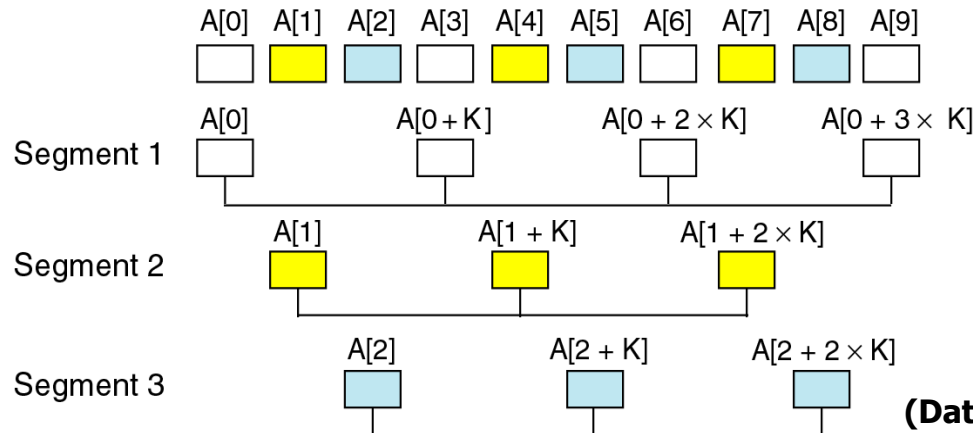
Analysis: inputs are presorted (best case inputs)

- The input is already sorted in increasing order
 - ♦ When inserting $A[P]$ into the sorted $A[0..P-1]$, only need to compare $A[P]$ with $A[P-1]$ and there is no data movement.
 - ♦ For each iteration of the **outer for-loop**, **the inner for-loop** terminates after checking the loop condition once $\Rightarrow O(N)$ time, which is the best case time complexity.
- If input is nearly sorted, insertion sort runs fast.

Shellsort

Shellsort

- Shellsort (inventor: Donald Shell, 1959) – diminishing increment sort
 - ♦ Comparing elements that are distant;
 - ♦ The distance between comparisons decreases until the adjacent elements
- Increment sequence: $h_1, h_2, h_3, \dots, h_t$ and $h_1=1$
- h_k -sorted: all elements spaced h_k apart are sorted
- An important property (state without proof)
 - ♦ An h_k -sorted file that is then h_{k-1} -sorted, remains h_k -sorted



$N=10, h_k=3$

(Data Structures: A Pseudocode Approach with C)

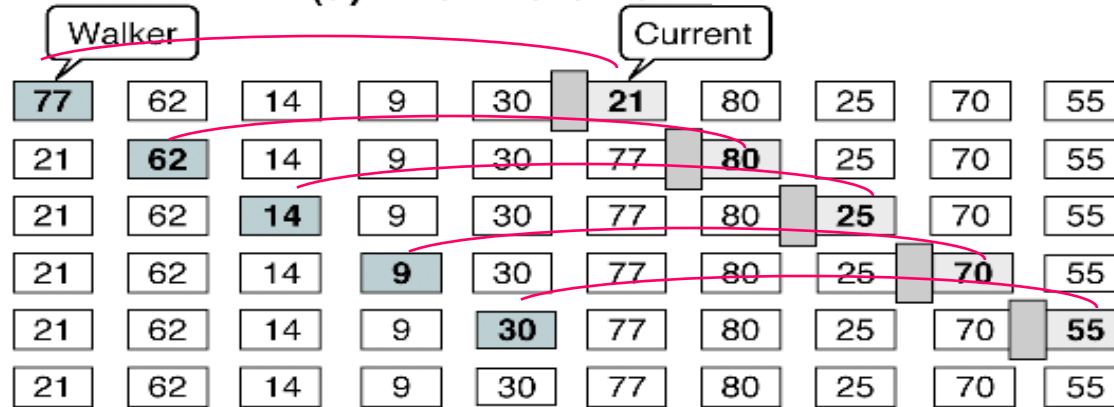
Shellsort Example

- 77, 62, 14, 9, 30, 21, 80, 25, 70, 55

Pass 1

(a) First increment:

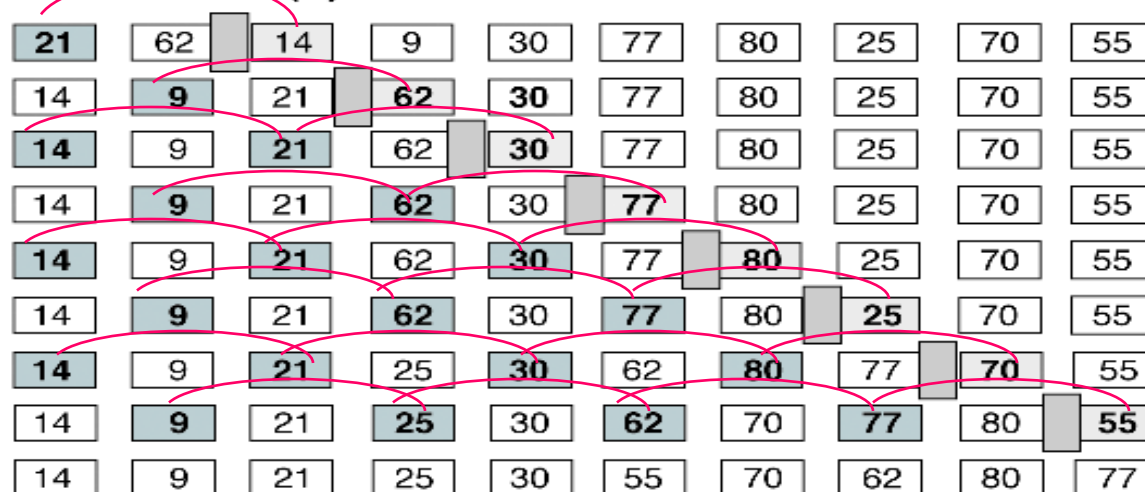
$h_3=5$



Pass 2

(b) Second increment:

$h_2=2$



Shellsort Example

- 77, 62, 14, 9, 30, 21, 80, 25, 70, 55

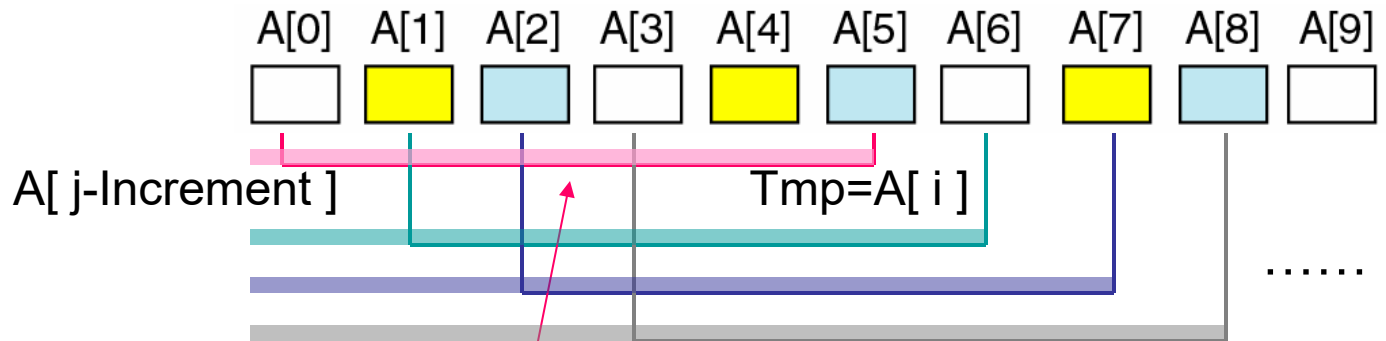
Pass 3 $h_1=1$

(c) Third increment:

14		9	21	25	30	55	70	62	80	77
9	14		21	25	30	55	70	62	80	77
9	14	21		25	30	55	70	62	80	77
9	14	21	25		30	55	70	62	80	77
9	14	21	25	30		55	70	62	80	77
9	14	21	25	30	55		70	62	80	77
9	14	21	25	30	55	70		62	80	77
9	14	21	25	30	55	62	70		80	77
9	14	21	25	30	55	62	70	80		77
9	14	21	25	30	55	62	70	77	80	

Shellsort

- Shell's increment: A popular (but poor) choice for increment sequence
- $h_t = \text{floor}(N/2)$, and $h_k = \text{floor}(h_{k+1}/2)$
- Shellsort routine using Shell's increments (better increments are possible)



```

void
Shellsort( ElementType A[ ], int N )
{
    int i, j, Increment;
    ElementType Tmp;

    for( Increment = N / 2; Increment > 0; Increment /= 2 )
        for( i = Increment; i < N; i++ )
        {
            Tmp = A[ i ];
            for( j = i; j >= Increment; j -= Increment )
            {
                if( Tmp < A[ j - Increment ] )
                    A[ j ] = A[ j - Increment ];
                else
                    break;
            }
            A[ j ] = Tmp;
        }
    }

```

Do h_t - order.

swap

Analysis of Shellsort

- The running time of shellsort depends on the choice of increment sequence.
- The analysis of shellsort is still an open problem, except for the most trivial increment sequences.
- The worst-case running time of shellsort, using Shell's increments, is $\Theta(N^2)$.
- Shellsort is unstable.

Heapsort

(Please refer to Lec10. in detail)

Heapsort

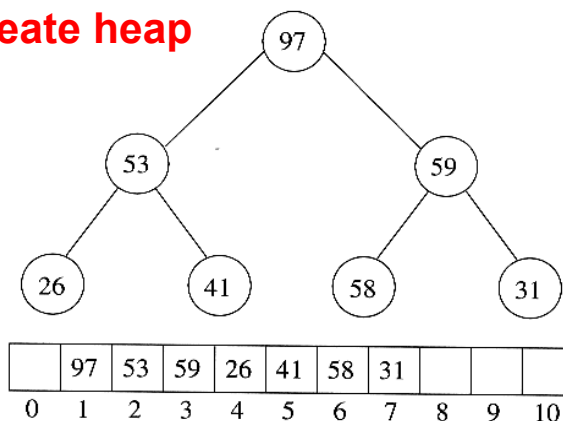
- The heapsort is an improved version of the selection sort in which the largest/smallest element (the root) is selected and exchanged with the last element in the unsorted list.
- Input: N elements
- Output: A heap with heap-order property
 - (1) Build a binary heap of N elements
 - the minimum element is at the top of the heap
 - (2) Perform N **DeleteMin** operations
 - the elements are extracted in sorted order
 - (3) Record these elements in a second array and then copy the array back.

Heapsort: No Extra Storage

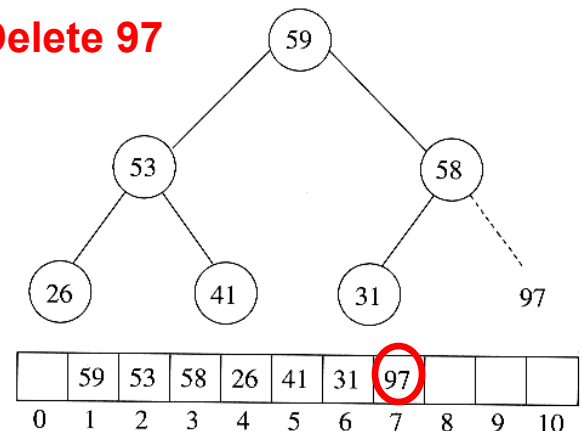
- After each **deleteMin**, the size of heap shrinks by 1
 - ♦ We can use **the last cell** just freed up to store the element that was just deleted
⇒ after the last **deleteMin**, the array will contain the elements in decreasing sorted order
- To sort the elements in the decreasing order, use a min heap
- To sort the elements in the increasing order, use a max heap
 - ♦ the parent has a larger element than the child

Create heap

Sort in increasing order: use max heap



Delete 97



Implementation of Heapsort

//adjust the elements in the array according to the property of the heap

```
#define LeftChild( i )  ( 2 * ( i ) + 1 )

void
PercDown( ElementType A[ ], int i, int N )
{
    int Child;
    ElementType Tmp;

    for( Tmp = A[ i ]; LeftChild( i ) < N; i = Child )
    {
        Child = LeftChild( i );
        if( Child != N - 1 && A[ Child + 1 ] > A[ Child ] )
            Child++;
        if( Tmp < A[ Child ] )
            A[ i ] = A[ Child ];
        else
            break;
    }
    A[ i ] = Tmp;
}
```

Implementation of Heapsort

Using Max Heap

```
void
Heapsort( ElementType A[ ], int N )
{
    int i;

    for( i = N / 2; i >= 0; i-- )    /* BuildHeap */
        PercDown( A, i, N );
    for( i = N - 1; i > 0; i-- )
    {
        Swap( &A[ 0 ], &A[ i ] );    /* DeleteMax */
        PercDown( A, 0, i );
    }
}
```

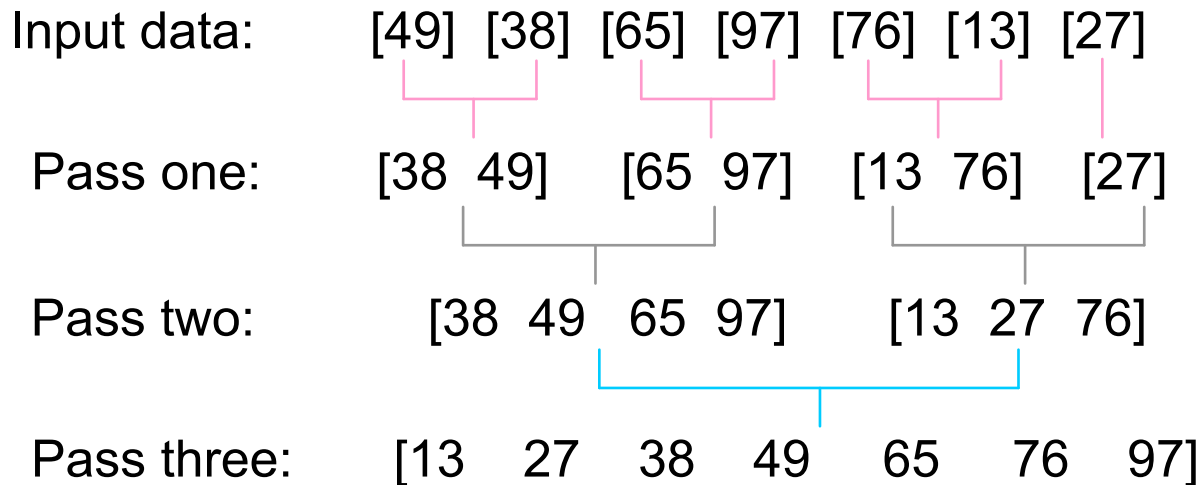
Heapsort – Running Time Analysis

- (1) Build a binary heap of N elements
 - ♦ Repeatedly insert N elements $\Rightarrow O(N \log N)$ time
 - (2) Perform N **DeleteMin** operations
 - ♦ Each **DeleteMin** operation takes $O(\log N) \Rightarrow O(N \log N)$
 - (3) Record these elements in a second array and then copy the array back
 - ♦ $O(N)$
- Total time complexity: $O(N \log N)$
 - Memory requirement: uses an extra array, $O(N)$

Mergesort

Mergesort

- The basic idea
 - ♦ Based on divide-and-conquer strategy
 - Firstly, divide the list into N parts, each has 1 element
 - Merge these sorted parts
 - Sort each list of the $\text{ceil}(N/2)$ parts
 - Merge these sorted parts
 - Loop, until the whole list is sorted



Mergesort (recursively)

- Divide the list into two smaller lists of about equal sizes
- Sort each smaller list *recursively*
- Merge the two sorted lists to get one sorted list

How do we divide the list? How much time is needed?

How do we merge the two sorted lists? How much time is needed?

How to divide?

- It is assumed that the input list is an array $A[0] \dots A[N-1]$.
- Dividing the array takes $O(1)$ time.
- We can represent a sublist by two integers **left** and **right**:
To divide it into two parts, we compute **center** = (**left** + **right**) / 2
and obtain **A[left] .. A[Center]** and **A[center+1] .. A[Right]**.

Mergesort Routine

- Divide-and-conquer strategy
 - ◆ Recursively mergesort the first half and the second half
 - ◆ Merge the two sorted halves together

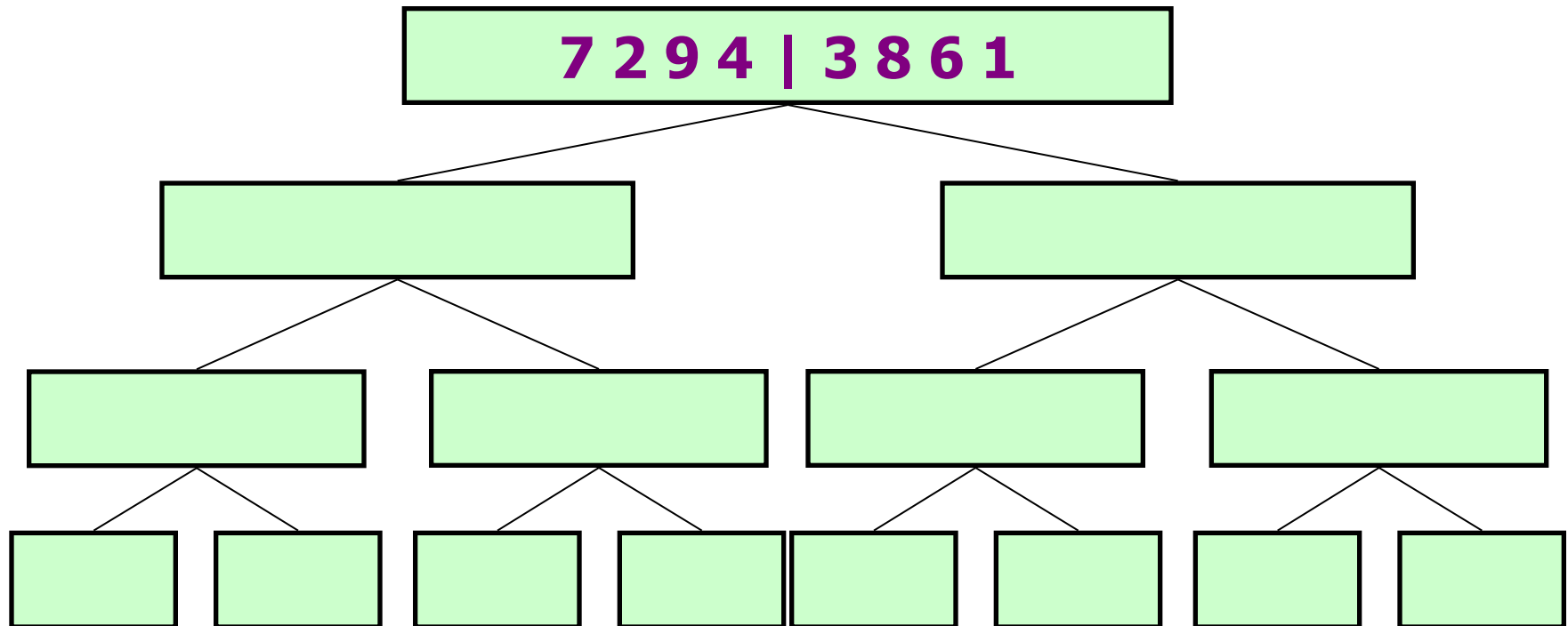
The main routine of merge sort

```
void
MSort( ElementType A[ ], ElementType TmpArray[ ],
      int Left, int Right )
{
    int Center;

    if( Left < Right )
    {
        Center = ( Left + Right ) / 2;
        MSort( A, TmpArray, Left, Center );
        MSort( A, TmpArray, Center + 1, Right );
        Merge( A, TmpArray, Left, Center + 1, Right );
    }
}
```

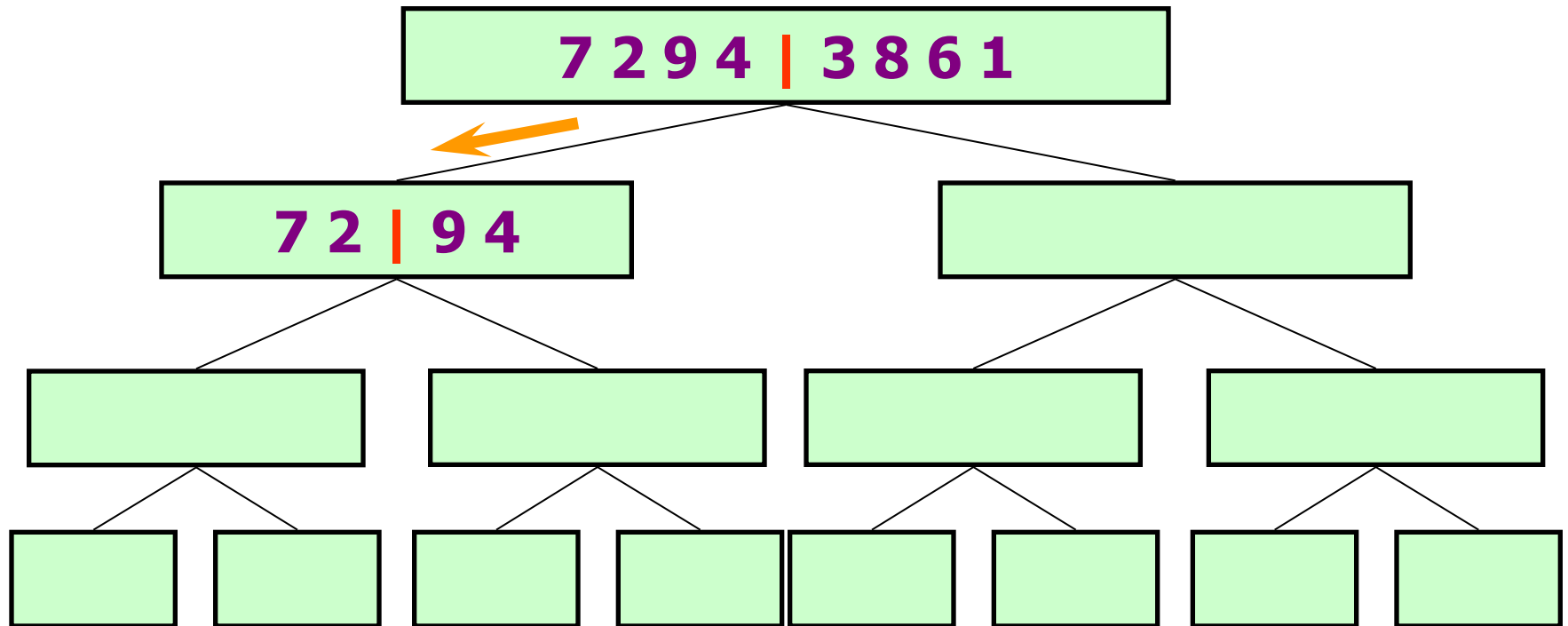
Mergesort Execution Example

Partition



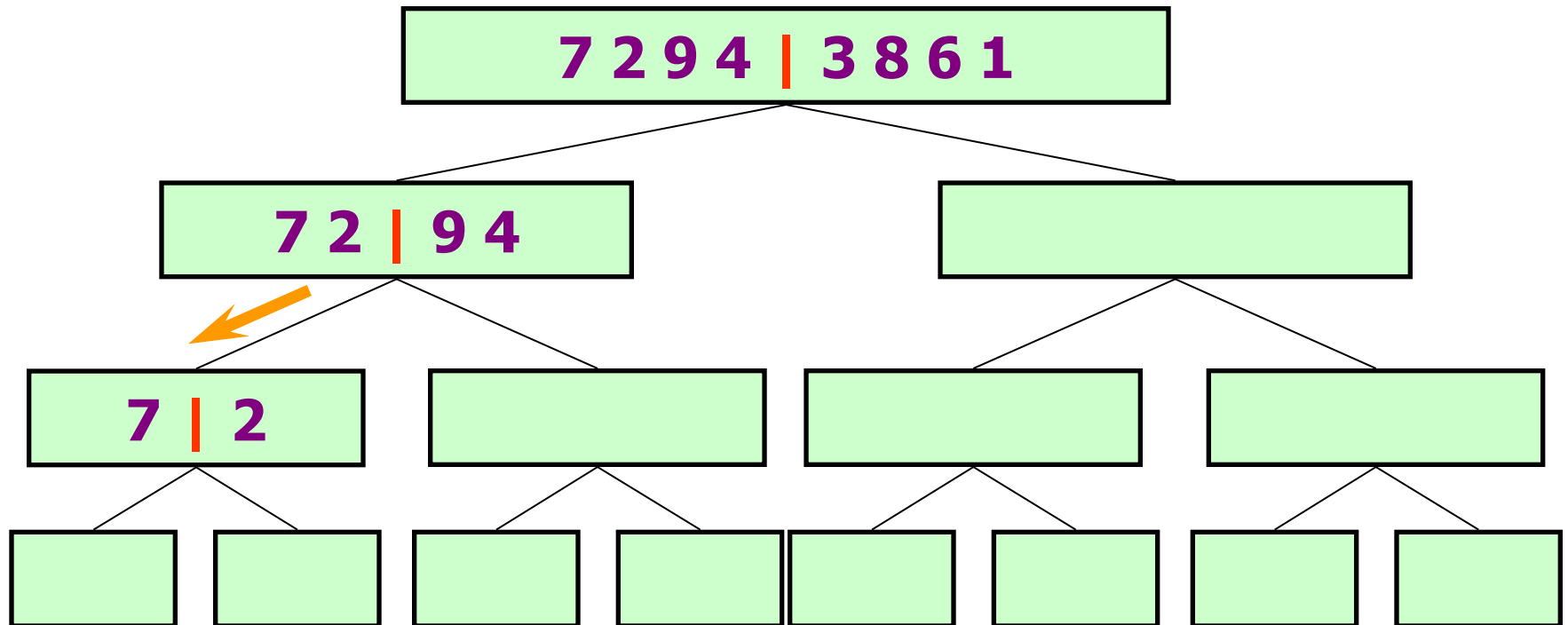
Mergesort Execution Example

Recursive call, partition



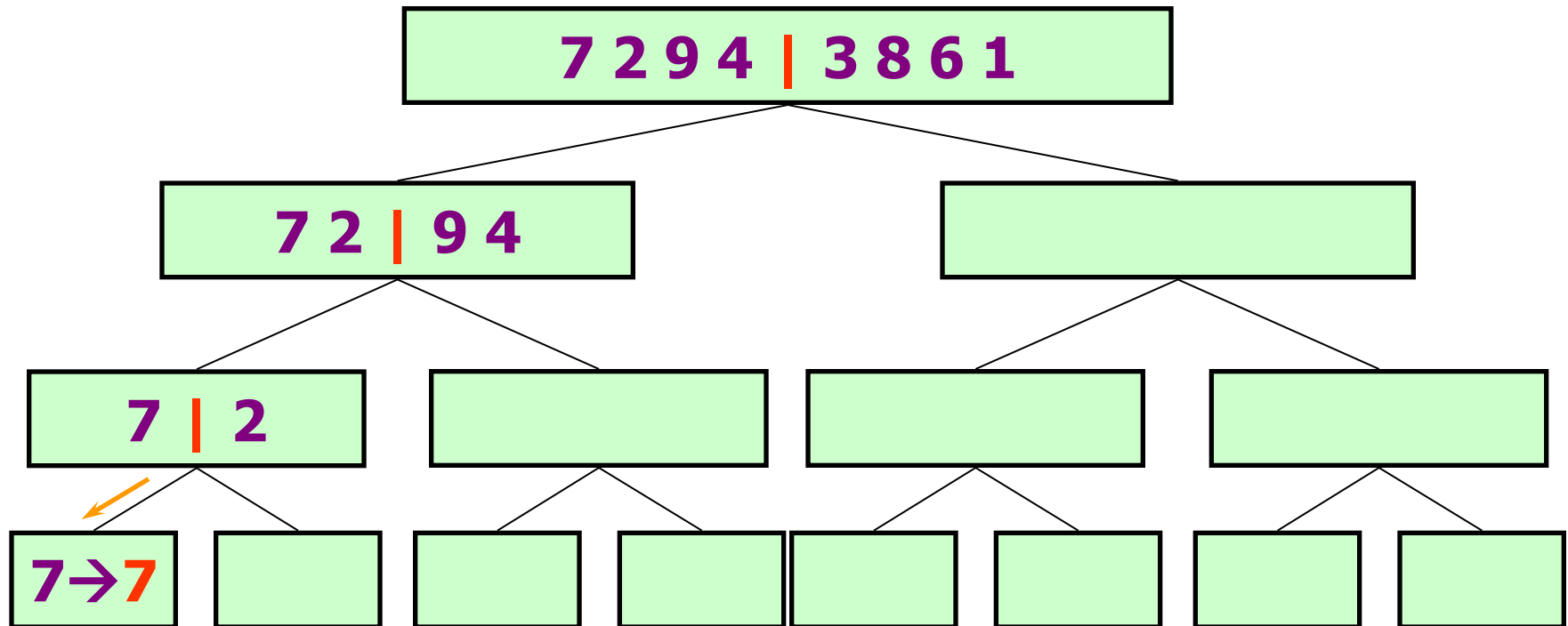
Mergesort Execution Example

Recursive call, partition



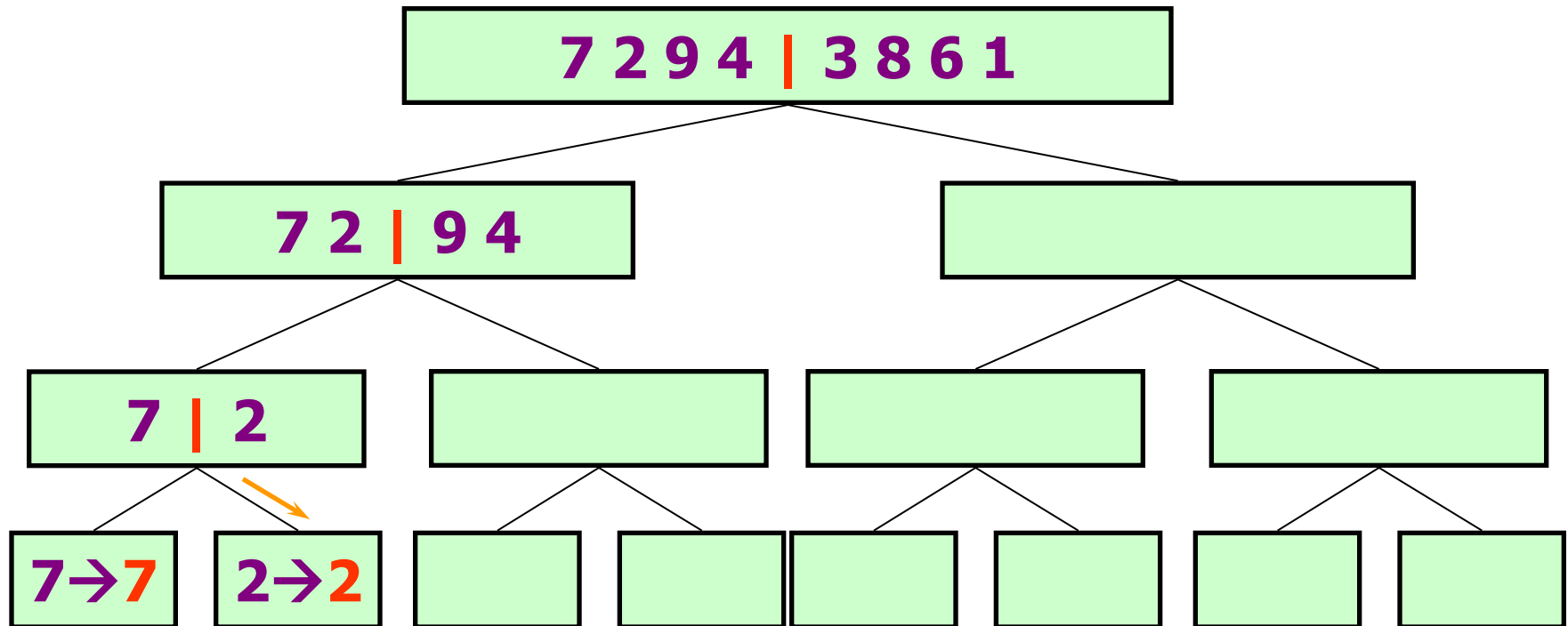
Mergesort Execution Example

Recursive call, base case



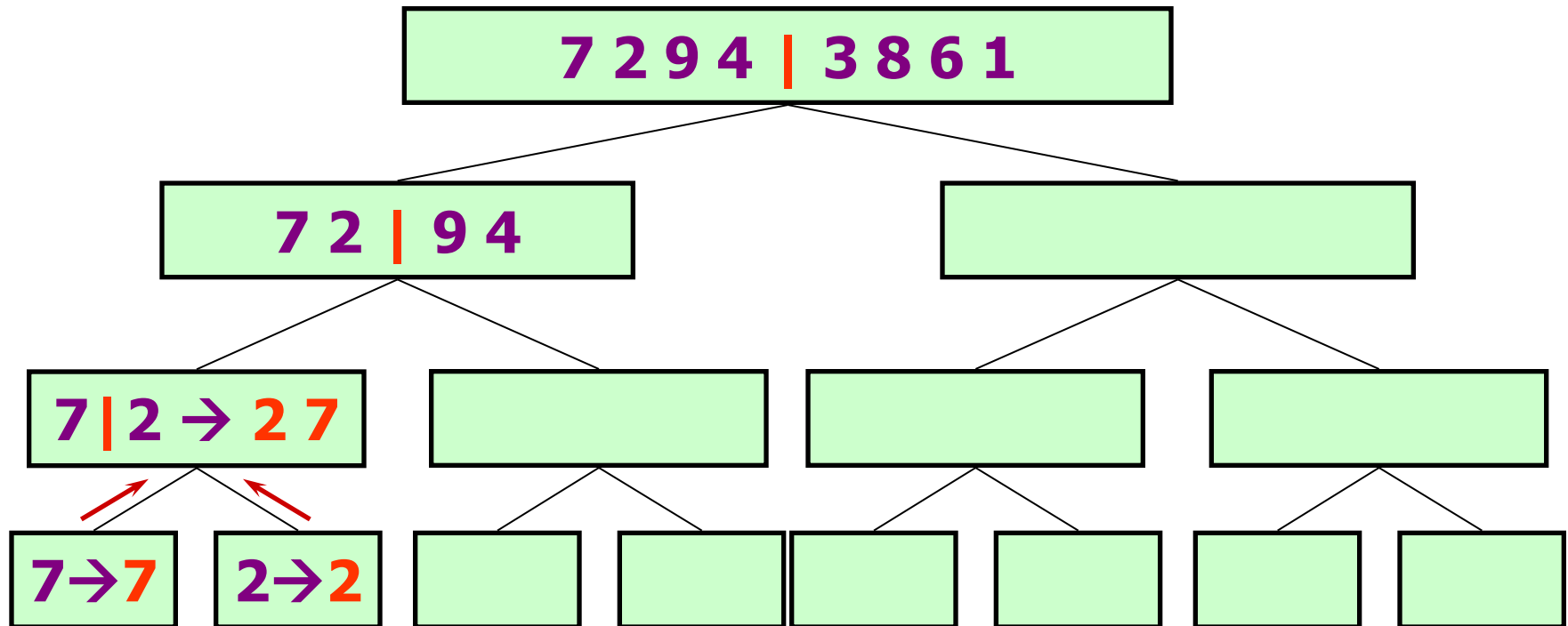
Mergesort Execution Example

Recursive call, base case



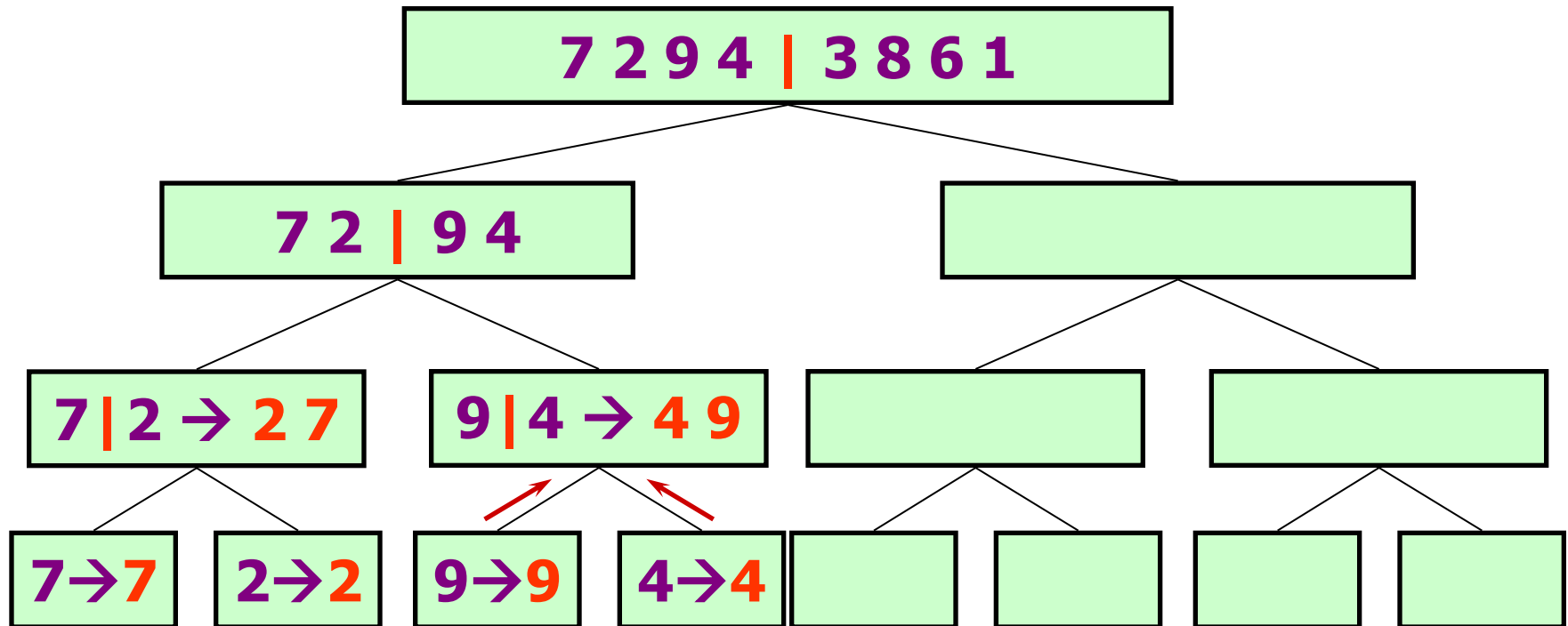
Mergesort Execution Example

Merge



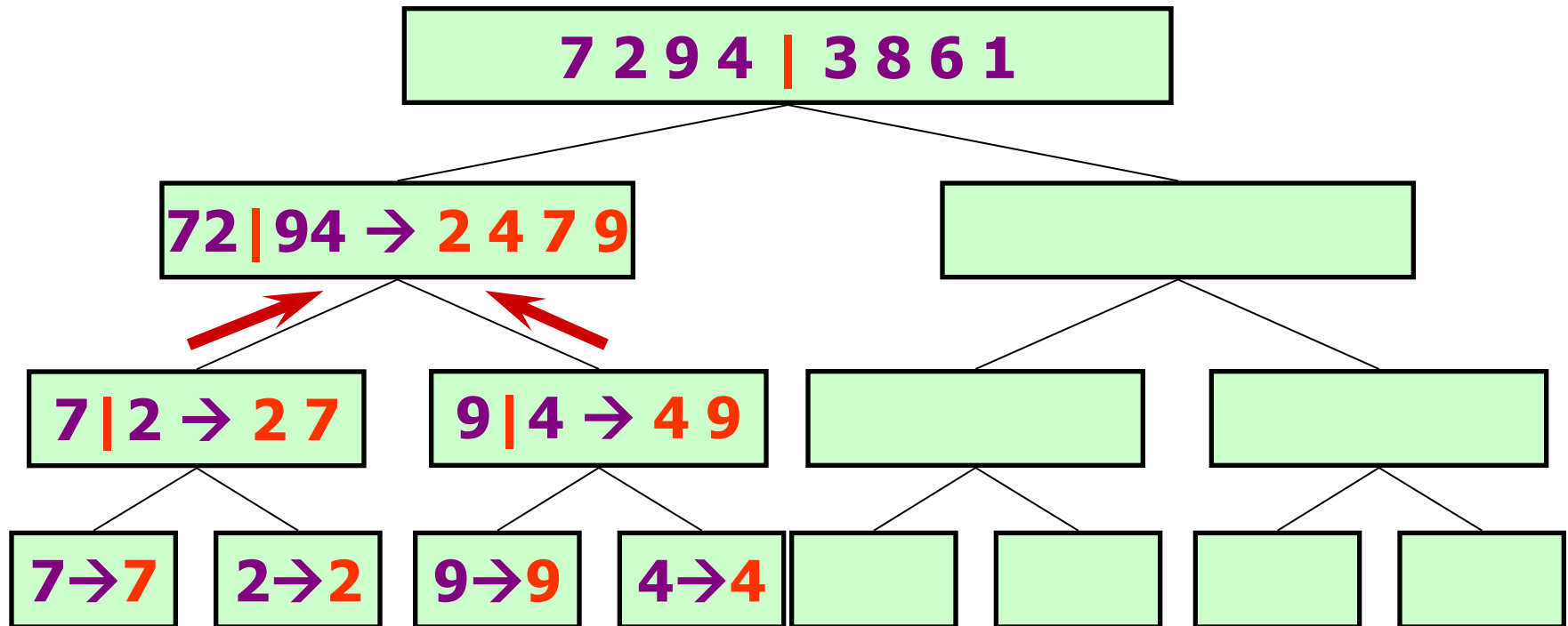
Mergesort Execution Example

Recursive call, base case, ..., base case, merge



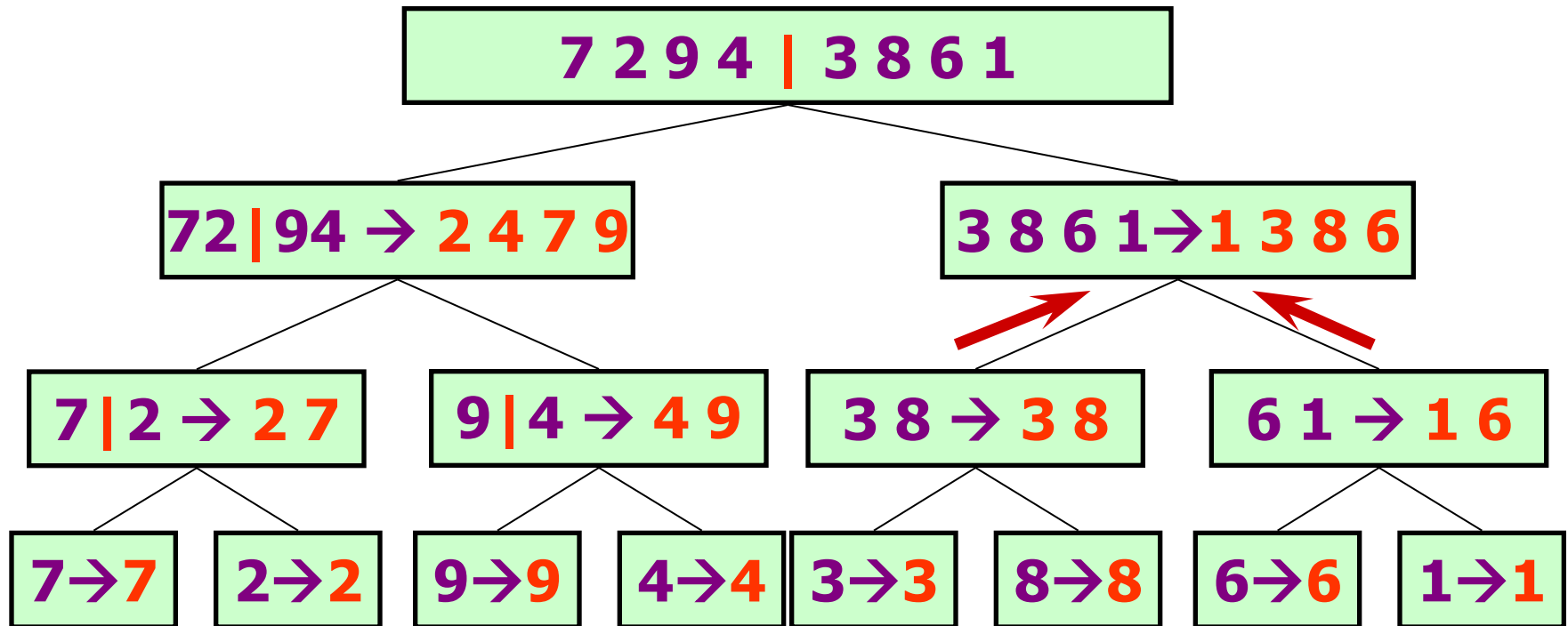
Mergesort Execution Example

Merge



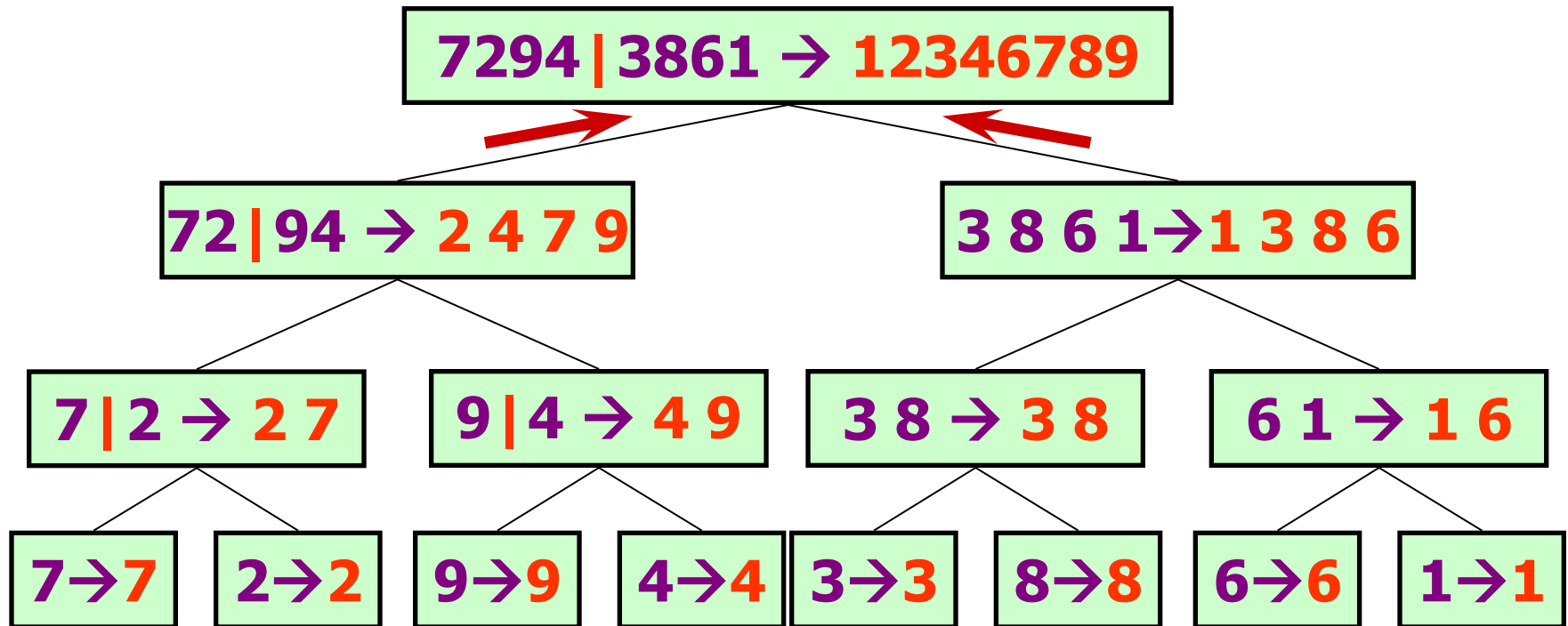
Mergesort Execution Example

Recursive call, ..., merge, merge



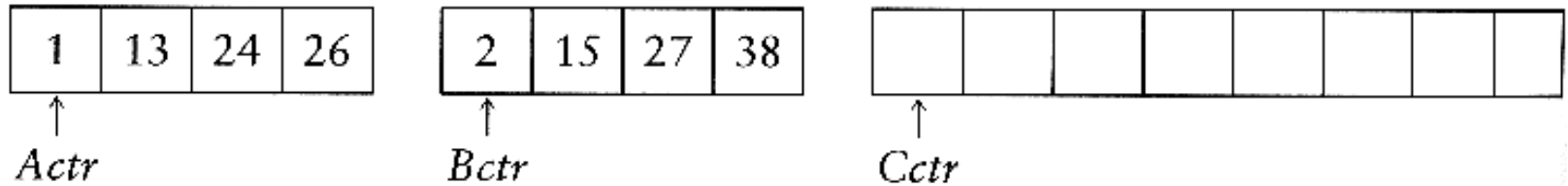
Mergesort Execution Example

Merge



How to Merge?

- Input: two sorted arrays A and B
- Output: an output sorted array C (Merged from A&B)
- Three counters: *Actr*, *Bctr*, and *Cctr*
 - ♦ initially set to the beginning of their respective arrays



- (1) The smaller of $A[Actr]$ and $B[Bctr]$ is copied to the next entry in C, and the appropriate counters are advanced;
- (2) When either input list is exhausted, the remainder of the other list is copied to C.

Implementation of Merging

The routine for merging

```
void
Merge( ElementType A[ ], ElementType TmpArray[ ],
      int Lpos, int Rpos, int RightEnd )
{
    int i, LeftEnd, NumElements, TmpPos;

    LeftEnd = Rpos - 1;
    TmpPos = Lpos;
    NumElements = RightEnd - Lpos + 1;

    /* main loop */
    while( Lpos <= LeftEnd && Rpos <= RightEnd )
        if( A[ Lpos ] <= A[ Rpos ] )
            TmpArray[ TmpPos++ ] = A[ Lpos++ ];
        else
            TmpArray[ TmpPos++ ] = A[ Rpos++ ];

    while( Lpos <= LeftEnd ) /* Copy rest of first half */
        TmpArray[ TmpPos++ ] = A[ Lpos++ ];
    while( Rpos <= RightEnd ) /* Copy rest of second half */
        TmpArray[ TmpPos++ ] = A[ Rpos++ ];

    /* Copy TmpArray back */
    for( i = 0; i < NumElements; i++, RightEnd-- )
        A[ RightEnd ] = TmpArray[ RightEnd ];
}
```

Implementation of Merge

The main routine of mergesort

```
void
MSort( ElementType A[ ], ElementType TmpArray[ ],
      int Left, int Right )
{
    int Center;

    if( Left < Right )
    {
        Center = ( Left + Right ) / 2;
        MSort( A, TmpArray, Left, Center );
        MSort( A, TmpArray, Center + 1, Right );
        Merge( A, TmpArray, Left, Center + 1, Right );
    }
}
```

Mergesort

Mergesort routine

```
void
Mergesort( ElementType A[ ], int N )
{
    ElementType *TmpArray;

    TmpArray = malloc( N * sizeof( ElementType ) );
    if( TmpArray != NULL )
    {
        MSort( A, TmpArray, 0, N - 1 );
        free( TmpArray );
    }
    else
        FatalError( "No space for tmp array!!!" );
}
```


Analysis of Mergesort

- It is a classic example of the techniques used to analyze recursive routines
 - ♦ Write a recurrence relation for the running time

Let $T(N)$ denote the worst-case running time of mergesort to sort N numbers.

Assume that N is a power of 2.

Recurrence for $T(N)$:

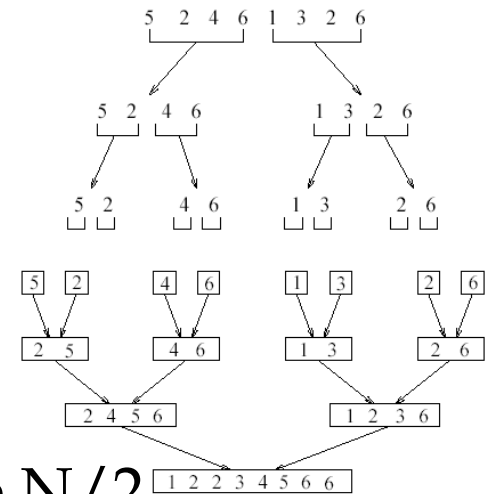
$$T(1) = 1, \text{ for } N=1$$

$$T(N) = 2T(N/2) + N$$

two recursive mergesorts of size $N/2$

+

the time to merge, which is linear: N



Analysis of Mergesort

$$T(1) = 1$$

$$T(N) = 2T(N/2) + N$$

This is a standard recurrence relation.

Solve the recurrence--Method 1:

$$\left. \begin{array}{l} \frac{T(N)}{N} = \frac{T(N/2)}{N/2} + 1 \\ \text{This is valid for any } N \text{ that is a power of 2, so} \\ \frac{T(N/2)}{N/2} = \frac{T(N/4)}{N/4} + 1 \\ \frac{T(N/4)}{N/4} = \frac{T(N/8)}{N/8} + 1 \\ \cdot \\ \cdot \\ \cdot \\ \frac{T(2)}{2} = \frac{T(1)}{1} + 1 \end{array} \right\} k$$

$$2^k = N \rightarrow k = \log N$$

$$\begin{array}{l} \text{Add up all.} \quad \frac{T(N)}{N} = \frac{T(1)}{1} + \log(N) \\ T(N) = N \log N + N = O(N \log N) \end{array}$$

- To be continued...
 - ♦ Exchange sort
 - Bubblesort
 - Quicksort
 - ♦ Other sorts